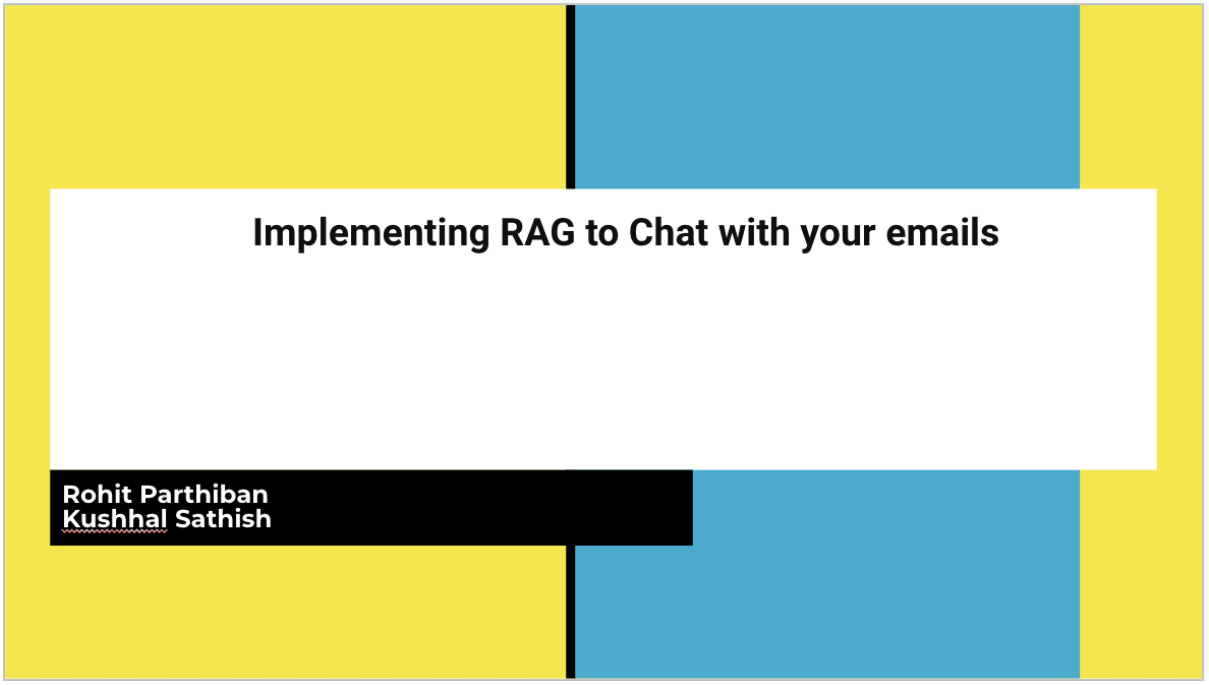
Santa Clara University

CSEN 280: Machine Learning

PROJECT REPORT



1. Abstract

This project integrates Retrieval-Augmented Generation (RAG) with a custom email data processing pipeline to enhance the accuracy and relevance of responses to user queries. Leveraging LangChain for Retreival, ChromaDB serves as the vector database to efficiently index and retrieve email content based on semantic similarity. The system incorporates a fine-tuned version of the Gemma language model, optimized using Databricks Dolly datasets, to align the model's outputs with domain-specific needs and enhance response generation quality. This hybrid approach of retrieval and fine-tuning enables the generation of contextually rich, precise, and tailored outputs, designed to streamline communication workflows and improve user satisfaction.

2. Problem Statement

Email communication is central to professional and personal correspondence, often serving as a repository of important discussions, information, and decision-making processes. However, retrieving relevant information or generating meaningful insights from extensive email threads can be challenging due to several factors:

1. **Volume of Data**: Users often deal with large volumes of email data, making it difficult to identify specific information quickly.
2. **Unstructured Format**: Emails contain unstructured text, attachments, metadata, and diverse formats, making information extraction and context understanding complex.
3. **Lack of Personalization**: Standard query-response systems may provide generic or irrelevant responses due to limited context awareness.
4. **Time Constraints**: Responding effectively to queries often requires significant manual effort to review past conversations and context.

3. Goals of AttendEase:

Efficient Information Retrieval:

- Enable users to quickly retrieve relevant information from large volumes of email data using semantic search, even when the query language is implicit or paraphrased.

Contextual Understanding:

- Develop a system that understands the context of user queries and matches it accurately with relevant email threads, metadata, or historical conversations.

Personalized and Accurate Response Generation:

- Generate human-like, natural, and context-aware responses that are tailored to the user's specific query, minimizing the need for manual intervention.

Optimization for Domain-Specific Use:

- Fine-tune the language model to align with the specific nuances of email communication, including tone, structure, and common patterns, for enhanced response quality.

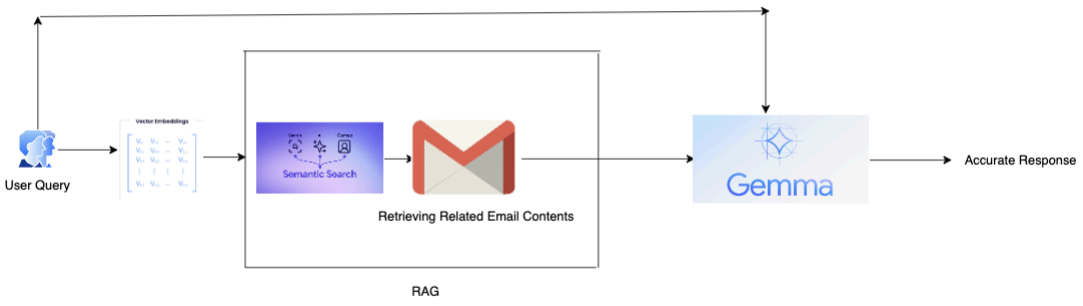
Seamless User Experience:

- Design a system that integrates seamlessly with email workflows, providing quick, reliable, and actionable insights without disrupting the user's productivity.

Scalability and Robustness:

Ensure the system can handle large datasets efficiently, scaling with user needs while maintaining high accuracy and performance.

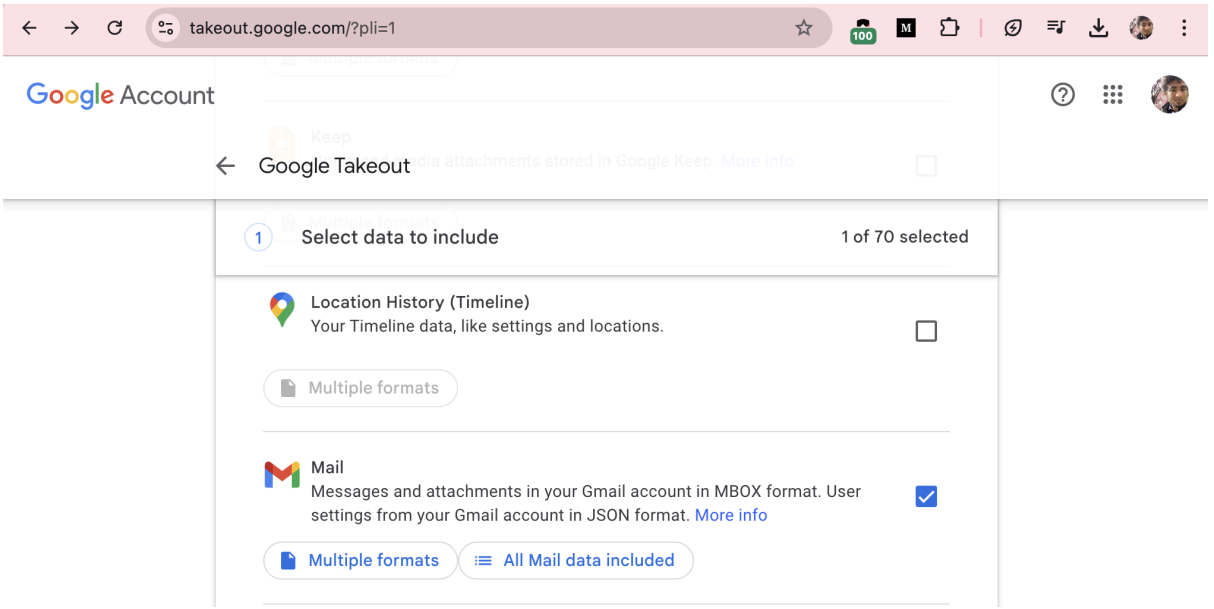
4. System Overview and Architecture



The project comprises Several main components: Dataset generation(Fine Tuning Gemma RAG retrieval) , Preprocessing Email Digests, convert text document to langchain document format, Fine Tuning the Gemma Model, Quantization of the model,RAG Retrieval of Documents-LLM response Generation.Each component plays a vital role in the overall functionality of the system.

Dataset Generation (Email Data)

Export Gmail data via Google Takeout as an .mbox file.
Parse the file using Python to extract email fields like subject, content, and metadata.Clean and structure the data into a usable format (e.g., JSON or CSV).
Convert email content to embeddings and index them in ChromaDB for semantic search.
Use the indexed data in your RAG pipeline to retrieve relevant emails and generate context-aware responses with the Gemma LLM.



Dataset Generation for Fine-tuning Gemma:

The code begins by loading the Databricks Dolly-15K dataset and analyzing the distribution of categories, such as **`closed_qa`** and **`classification`**. It then filters out entries that lack the **`context`** field, ensuring only examples with valid **`instruction`** and **`response`** fields are retained. The remaining data is reformatted into a structured text format, suitable for fine-tuning or downstream processing. This filtered dataset is saved as a JSONL file for easy access and reuse. Additionally, a smaller subset of the dataset, containing 1,000 rows, is loaded for experimentation or quick testing purposes.

```
[ ] from datasets import load_dataset

    dataset_name = "databricks/databricks-dolly-15k"
    dataset = load_dataset(dataset_name, split="train")
```

```
    from collections import defaultdict

    categries_count = defaultdict(int)
    for __, data in enumerate(dataset):
        categries_count[data['category']] += 1
    print(categries_count)
```

```
defaultdict(<class 'int'>, {'closed_qa': 1773, 'classification': 2136, 'open_qa': 3742, 'information_extraction': 1506, 'brainstorming': 1
```

```
[ ] # filter out those that do not have any context
    filtered_dataset = []
    for __, data in enumerate(dataset):
        if data["context"]:
            continue
        else:
            text = f"Instruction:\n{data['instruction']}\n\nResponse:\n{data['response']}"
            filtered_dataset.append({"text": text})

    print(filtered_dataset[0:2])
```

```
[{'text': 'Instruction:\nWhich is a species of fish? Tope or Rope\n\nResponse:\nTope'}, {'text': 'Instruction:\nWhy can camels survive for
```

```
[ ] # convert to json and save the filtered dataset as jsonl file
    import jsonlines as jl
    with jl.open('dolly-mini-train.jsonl', 'w') as writer:
        writer.write_all(filtered_dataset[0:])
```

```
[ ] from datasets import load_dataset

    dataset_name = "Kushhal/databricks-mini"
    dataset = load_dataset(dataset_name, split="train[0:1000]")
    dataset
```

```
Dataset({
    features: ['text'],
    num_rows: 1000
})
```

Preprocess the Email Digests

The code processes an MBOX file containing emails, extracting and parsing their content. It defines a class `MboxReader` to read and iterate over the MBOX file, where emails are identified by lines starting with "From ". Using this class, the code reads the file, decodes the email payloads, and extracts text content from HTML emails using BeautifulSoup. The process limits the number of emails processed (in this case, 10) and concatenates the cleaned text content into a single string for further use. This approach efficiently handles email data parsing and extraction for downstream tasks.

```
[ ]
    class MboxReader:
        def __init__(self, filename):
            self.handle = open(filename, 'rb')
            assert self.handle.readline().startswith(b'From ')

        def __enter__(self):
            return self

        def __exit__(self, exc_type, exc_value, exc_traceback):
            self.handle.close()

        def __iter__(self):
            return iter(self.__next__())

        def __next__(self):
            lines = []
            while True:
                line = self.handle.readline()
                if line == b'' or line.startswith(b'From '):
                    yield email.message_from_bytes(b''.join(lines), policy=default)
                    if line == b'':
                        break
                    lines = []
                    continue
                lines.append(line)
```

```
[ ]  path = "./Takeout/Mail/Arxiv.mbox"
     mbox = MboxReader(path)
     emails_to_process = 10

     current_mails = 0
     arxiv_contents = ""
     for __, message in tqdm(enumerate(mbox)):
         payload = message.get_payload(decode=True)
         if payload:
             current_mails += 1
             if current_mails > emails_to_process:
                 break
             soup = BeautifulSoup(payload, 'html.parser')
             body_text = soup.get_text().replace('"','').replace("\n", "").replace("\t", "").strip()
             arxiv_contents += body_text + " "
```

Fine Tuning the Gemma Model.

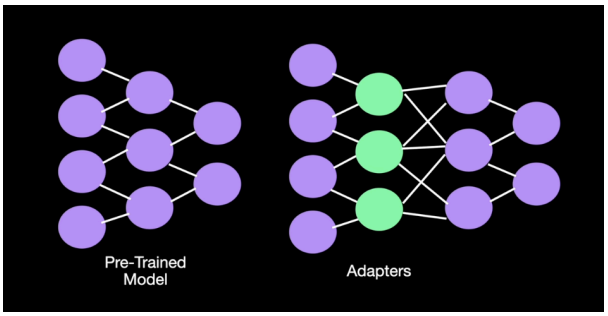
GPT4All embeddings provide an efficient, privacy-conscious, and accessible solution for semantic understanding and retrieval tasks. Their local processing capabilities make them ideal for scenarios requiring sensitive data handling, while their compatibility with tools like ChromaDB enables seamless integration into AI workflows. These embeddings empower developers to build robust and scalable AI solutions without heavy infrastructure requirements.

```
doc = Document(page_content=arxiv_contents,
               metadata={"source": "local"})

# split into different chunks
# chunk_size and chunk_overlap are a hyperparameters we choose
text_splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=20)

# split the documents and convert to vector stores
all_splits = text_splitter.split_documents([doc])
vector_store = Chroma.from_documents(documents=all_splits,
                                     embedding=GPT4AllEmbeddings())
```

```
Downloading: 100%|| 45.9M/45.9M [00:02<00:00, 18.4M
Verifying: 100%|| 45.9M/45.9M [00:00<00:00, 399M
```

Using LoRA

We are using **LoRA(Low-Rank Adaptation)**, a parameter-efficient fine-tuning method used to adapt large pre-trained language models like Gemma for specific tasks. LoRA works by freezing the original model weights and injecting trainable low-rank matrices into the model, significantly reducing the number of parameters to optimize during fine-tuning. This technique allows effective task-specific adaptation without retraining the entire model, making it both computationally efficient and memory-friendly. In the code, LoRA layers are likely applied to specific parts of the model (e.g., attention layers), enabling the fine-tuning process to focus on task-relevant updates while retaining the general capabilities of the pre-trained model. This is particularly useful for adapting Gemma to domain-specific tasks like email data processing while preserving the efficiency of fine-tuning.



LoRA Configurations

```
################################################
# LoRA parameters
################################################
# LoRA attention dimension
# lora_r = 64
lora_r = 4
# Alpha parameter for LoRA scaling
lora_alpha = 16
# Dropout probability for LoRA layers
lora_dropout = 0.1
```

```
# Load LoRA configuration
peft_config = LoraConfig(
    lora_alpha=lora_alpha,
    lora_dropout=lora_dropout,
    r=lora_r,
    bias="none",
    task_type="CAUSAL_LM",
    target_modules=["q_proj", "k_proj", "v_proj", "o_proj","gate_proj", "up_proj"]
)
```

```
[ ] # Train model
    trainer.train()
    trainer.model.save_pretrained(new_model)
    model = trainer.model.base_model
    model.save_pretrained("./merge_model")
```

[670/670 09:26, Epoch 1/1]

Step	Training Loss
25	4.835400
50	3.207800
75	3.097300
100	2.875100
125	2.813000
150	2.854800
175	2.766900
200	2.757500
225	2.738700
250	2.832300
275	2.663700
300	2.681500
325	2.647600
350	2.768700
375	2.683100
400	2.605700
425	2.745600
450	2.658900
475	2.614700
500	2.703300
525	2.642700
550	2.639100
575	2.704800
600	2.625900
625	2.715500
650	2.699000

Prompt the newly fine-tuned model

Load and MERGE the LoRA weights with the model weights

Run inference with the same prompt we used to test the pre-trained model

```
[ ] input_text = "What should I do on a trip to Europe?"

    base_model = AutoModelForCausalLM.from_pretrained(
        model_name,
        low_cpu_mem_usage=True,
        return_dict=True,
        torch_dtype=torch.float16,
        device_map=device_map,
    )
    model = PeftModel.from_pretrained(base_model, new_model)
    model = model.merge_and_unload()

    # Reload tokenizer to save it
    tokenizer = AutoTokenizer.from_pretrained(model_name, trust_remote_code=True)
    tokenizer.pad_token = tokenizer.eos_token
    tokenizer.padding_side = "right"
```

Quantization is Applied in `llama.cpp`

1. **Load Pre-trained Model**:
    - A large pre-trained language model (e.g., LLaMA) is loaded in its original precision (usually FP16 or FP32).
2. **Quantization Process**:
    - The weights of the model are quantized to lower-bit formats (e.g., INT8, INT4).
    - `llama.cpp` implements efficient quantization schemes such as:
        - **Q4_0 and Q4_1**: Fixed quantization levels.
        - **Dynamic Quantization**: Adapts quantization scales for better accuracy.
    - These techniques ensure that the model remains capable of generating high-quality outputs while reducing size and resource requirements.
3. **Storage and Execution**:
    - The quantized model is saved in a format optimized for fast loading and execution.
    - At runtime, `llama.cpp` leverages CPU-specific optimizations to perform inference without requiring GPUs, making it accessible to users with limited hardware resources.

Response without RAG just using the LLM model:

LLM response without RAG

```
[ ] # retrieve relevant docs
    rag_prompt = hub.pull("rlm/rag-prompt")
    retriever = vector_store.as_retriever()

    # Create the langchain with retriever
    qa_chain = (
        {"context": {}, "question": RunnablePassthrough()}
        | rag_prompt
        | llm
        | StrOutputParser()
    )
    qa_chain.invoke("what is the title of the paper that talks about awareness")
```

```
' There is no specific information provided in the context to determine the title of a paper.'
```

Response with RAGl:

LLM response with RAG

```
[ ] # retrieve relevant docs
    rag_prompt = hub.pull("rlm/rag-prompt")
    rag_prompt.messages
```

```
[HumanMessagePromptTemplate(prompt=PromptTemplate(input_variables=['context', 'question'], template="You are an assistant for question-answering tasks. Use the following pieces of retrieved context to answer the question. If you don't know the answer, just say that you don't know. Use three sentences maximum and keep the answer concise.\nQuestion: {question} \nContext: {context} \nAnswer:"))]
```

```
[ ] def format_documents(documents):
        return "\n\n".join(documents.page_content for doc in docs)
```

```
[ ] retriever = vector_store.as_retriever()

    # Create the langchain with retriever,
    # prompt template and LLM
    qa_chain = (
        {"context": retriever | format_documents, "question": RunnablePassthrough()}
        | rag_prompt
        | llm
        | StrOutputParser()
    )
    qa_chain.invoke("what is the title of the paper that talks about awareness?")
```

```
' The title of the paper that talks about awareness is "I Think, Therefore I am: Awareness in Large Language Models".'
```

Conclusion

This project successfully integrates advanced AI techniques to enhance email query handling through a robust Retrieval-Augmented Generation (RAG) pipeline. By leveraging ChromaDB for semantic search and embedding retrieval, and a fine-tuned Gemma LLM trained on the Databricks Dolly dataset, the system delivers accurate, context-aware, and human-like responses to user queries. The use of Google Takeout data for real-world email processing, combined with techniques like LoRA for efficient fine-tuning and model quantization with llama.cpp, ensures scalability, cost-effectiveness, and privacy preservation.

The seamless orchestration of components using LangChain allows for modular and adaptable workflows, while the integration of lightweight and efficient technologies such as GPT4All embeddings provides flexibility for deployment on resource-constrained devices. Overall, this system addresses the challenges of managing unstructured email data and enables users to efficiently retrieve and interact with meaningful insights, reducing cognitive load and enhancing productivity.

This project showcases the potential of combining retrieval-based approaches with fine-tuned language models to tackle domain-specific challenges. By bridging semantic understanding, efficient retrieval, and natural language generation, it sets the foundation for future applications in email automation, document management, and beyond.